WEEK-8

OSSP - SKILL Content Delivery Concepts
Course Code: 25CS2104E

TASK 1

Objective / Skill:

To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency.

Implementation / Work:

```
/*
 * Week 8 - Task 1: Command History Buffer
 * Objective: To Store Commands, Maintain History Buffer, Handle Capacity Limits,
 *            Retrieve Entries, Display History, Validate Consistency.
 */
#include <stdio.h>
#include <string.h>

#define HISTORY_SIZE 5
#define MAX_LEN 64

typedef struct {
    char entries[HISTORY_SIZE][MAX_LEN];
    int count;      /* entries currently stored            */
    int total;      /* commands ever added (absolute no.)  */
} History;

void history_add(History *h, const char *cmd) {
    if (h->count == HISTORY_SIZE) {            /* capacity reached: drop oldest */
        for (int i = 1; i < HISTORY_SIZE; i++)
            strcpy(h->entries[i - 1], h->entries[i]);
        h->count--;
    }
    strncpy(h->entries[h->count], cmd, MAX_LEN - 1);
    h->entries[h->count][MAX_LEN - 1] = '\0';
    h->count++;
    h->total++;
}

const char *history_get(History *h, int number) {
    int first = h->total - h->count + 1;       /* absolute number of oldest entry */
    if (number < first || number > h->total) return NULL;
    return h->entries[number - first];
}

void history_display(History *h) {
    int first = h->total - h->count + 1;
    for (int i = 0; i < h->count; i++)
        printf("  %3d  %s\n", first + i, h->entries[i]);
}

int history_validate(History *h) {
    if (h->count < 0 || h->count > HISTORY_SIZE) return 0;
    if (h->total < h->count) return 0;
    for (int i = 0; i < h->count; i++)
        if (h->entries[i][0] == '\0') return 0;
    return 1;
}

int main() {
    printf("=== Week 8 Task 1: Command History Buffer ===\n\n");
    History h = {0};
    const char *cmds[] = {"ls -l", "pwd", "cd /tmp", "cat notes.txt",
                          "grep -i ossp notes.txt", "echo hello", "date"};
    int n = sizeof(cmds) / sizeof(cmds[0]);

    printf("History capacity: %d entries\n\n", HISTORY_SIZE);
    for (int i = 0; i < n; i++) {
        history_add(&h, cmds[i]);
        printf("[Added #%d] %-24s (stored: %d/%d)\n", h.total, cmds[i], h.count, HISTORY_SIZE);
    }

    printf("\nExecuting 'history' command:\n");
    history_display(&h);
```

```
    printf("\nRetrieving entry 5 (!5): ");
    const char *e = history_get(&h, 5);
    printf("%s\n", e ? e : "event not found");

    printf("Retrieving entry 1 (!1): ");
    e = history_get(&h, 1);
    printf("%s\n", e ? e : "event not found (overwritten after capacity limit)");

    printf("\nConsistency check: %s\n", history_validate(&h) ? "PASSED" : "FAILED");
    printf("History buffer management verified successfully!\n");
    return 0;
}
```

Output:

```
charith@DESKTOP-RBQQSS:~$ nano s8.1.c
charith@DESKTOP-RBQQSS:~$ gcc s8.1.c -o s8.1
charith@DESKTOP-RBQQSS:~$ ./s8.1
=== Week 8 Task 1: Command History Buffer ===

History capacity: 5 entries

[Added #1] ls -l                    (stored: 1/5)
[Added #2] pwd                      (stored: 2/5)
[Added #3] cd /tmp                  (stored: 3/5)
[Added #4] cat notes.txt            (stored: 4/5)
[Added #5] grep -i ossp notes.txt   (stored: 5/5)
[Added #6] echo hello               (stored: 5/5)
[Added #7] date                     (stored: 5/5)

Executing 'history' command:
    3  cd /tmp
    4  cat notes.txt
    5  grep -i ossp notes.txt
    6  echo hello
    7  date

Retrieving entry 5 (!5): grep -i ossp notes.txt
Retrieving entry 1 (!1): event not found (overwritten after capacity limit)

Consistency check: PASSED
History buffer management verified successfully!
charith@DESKTOP-RBQQSS:~$
```

TASK 2

Objective / Skill:

To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations.

Implementation / Work:

```
/*
 * Week 8 - Task 2: Pipeline Structures and Layout Validation
 * Objective: To Create Pipeline Structures, Store Commands, Maintain Execution Order,
 *            Connect Processes, Validate Pipeline Layout, Test Configurations.
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_ARGS 16
#define MAX_COMMANDS 8

typedef struct {
    char *args[MAX_ARGS];
    int argc;
} Command;

typedef struct {
```

```
    Command commands[MAX_COMMANDS];
    int command_count;
} Pipeline;

int validate_pipeline_layout(const char *cmd_line) {
    if (!cmd_line || strlen(cmd_line) == 0) return -1;
    // Check leading or trailing pipe
    while (*cmd_line == ' ') cmd_line++;
    if (*cmd_line == '|') {
        printf("  [Validation Error] Syntax error: Leading pipe '|' detected.\n");
        return -1;
    }
    size_t len = strlen(cmd_line);
    if (len > 0 && cmd_line[len - 1] == '|') {
        printf("  [Validation Error] Syntax error: Trailing pipe '|' detected.\n");
        return -1;
    }
    // Check consecutive pipes "||"
    if (strstr(cmd_line, "||") != NULL) {
        printf("  [Validation Error] Syntax error: Consecutive pipes '||' detected.\n");
        return -1;
    }
    return 0;
}

void parse_to_pipeline(char *line, Pipeline *pl) {
    pl->command_count = 0;
    char *save_outer, *save_inner;
    char *token = strtok_r(line, "|", &save_outer);
    while (token && pl->command_count < MAX_COMMANDS) {
        Command *cmd = &pl->commands[pl->command_count++];
        cmd->argc = 0;
        char *sub = strtok_r(token, " \t\n", &save_inner);
        while (sub && cmd->argc < MAX_ARGS - 1) {
            cmd->args[cmd->argc++] = sub;
            sub = strtok_r(NULL, " \t\n", &save_inner);
        }
        cmd->args[cmd->argc] = NULL;
        token = strtok_r(NULL, "|", &save_outer);
    }
}

int main() {
    printf("=== Week 8 Task 2: Pipeline Structures & Validation ===\n\n");

    char test1[] = "cat input.txt | grep -i error | sort | wc -l";
    char test2[] = "| ls -la | wc -l";
    char test3[] = "cat file.txt || sort";

    printf("[Test Config 1] Validating layout: '%s'\n", test1);
    if (validate_pipeline_layout(test1) == 0) {
        printf("  Layout Status: VALID. Storing commands in execution order:\n");
        Pipeline pl;
        parse_to_pipeline(test1, &pl);
        for (int i = 0; i < pl.command_count; i++) {
            printf("    Stage [%d]: Command '%s' with %d arguments.\n",
                   i + 1, pl.commands[i].args[0], pl.commands[i].argc);
        }
    }

    printf("\n[Test Config 2] Validating layout: '%s'\n", test2);
    validate_pipeline_layout(test2);

    printf("\n[Test Config 3] Validating layout: '%s'\n", test3);
    validate_pipeline_layout(test3);

    printf("\nSUCCESS: Pipeline structures configured and validated successfully.\n");
    return 0;
}
```

Output:

```
charith@DESKTOP-RBQQSS:~$ nano s8.2.c
charith@DESKTOP-RBQQSS:~$ gcc s8.2.c -o s8.2
charith@DESKTOP-RBQQSS:~$ ./s8.2
=== Week 8 Task 2: Pipeline Structures & Validation ===

[Test Config 1] Validating layout: 'cat input.txt | grep -i error | sort | wc -l'
  Layout Status: VALID. Storing commands in execution order:
    Stage [1]: Command 'cat' with 2 arguments.
    Stage [2]: Command 'grep' with 3 arguments.
    Stage [3]: Command 'sort' with 1 arguments.
    Stage [4]: Command 'wc' with 2 arguments.

[Test Config 2] Validating layout: '| ls -la | wc -l'
  [Validation Error] Syntax error: Leading pipe '|' detected.

[Test Config 3] Validating layout: 'cat file.txt || sort'
  [Validation Error] Syntax error: Consecutive pipes '||' detected.

SUCCESS: Pipeline structures configured and validated successfully.
charith@DESKTOP-RBQQSS:~$ |
```

TASK 3

Objective / Skill:

To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow.

Implementation / Work:

```
/*
 * Week 8 - Task 3: Two-Stage Pipeline Execution
 * Objective: To Create Pipes, Redirect Streams, Execute Connected Commands,
 *            Close Descriptors, Synchronize Processes, Test Data Flow.
 */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int pipefd[2];
    printf("=== Week 8 Task 3: Two-Stage Pipeline Execution ===\n");
    printf("Executing pipeline: echo \"hello from ossp pipeline\" | tr a-z A-Z\n\n");
    fflush(stdout);

    if (pipe(pipefd) < 0) {
        perror("pipe creation failed");
        return 1;
    }

    pid_t p1 = fork();
    if (p1 == 0) {
        // Child 1: Producer (stdout -> pipe write end)
        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[0]);
        close(pipefd[1]);
        char *args[] = {"echo", "hello from ossp pipeline", NULL};
        execvp(args[0], args);
        perror("execvp child 1");
        exit(1);
    }

    pid_t p2 = fork();
    if (p2 == 0) {
        // Child 2: Consumer (stdin -> pipe read end)
        dup2(pipefd[0], STDIN_FILENO);
        close(pipefd[0]);
        close(pipefd[1]);
        char *args[] = {"tr", "a-z", "A-Z", NULL};
        execvp(args[0], args);
        perror("execvp child 2");
```

```
        exit(1);
    }

    // Parent: close descriptors and synchronize
    close(pipefd[0]);
    close(pipefd[1]);

    int status1, status2;
    waitpid(p1, &status1, 0);
    waitpid(p2, &status2, 0);

    printf("\nSynchronization complete. Both processes terminated cleanly.\n");
    printf("Pipeline stream redirection and data flow verified!\n");
    return 0;
}
```

Output:

```
charith@DESKTOP-RBQQSS:~$ nano s8.3.c
charith@DESKTOP-RBQQSS:~$ gcc s8.3.c -o s8.3
charith@DESKTOP-RBQQSS:~$ ./s8.3
=== Week 8 Task 3: Two-Stage Pipeline Execution ===
Executing pipeline: echo "hello from ossp pipeline" | tr a-z A-Z

HELLO FROM OSSP PIPELINE

Synchronization complete. Both processes terminated cleanly.
Pipeline stream redirection and data flow verified!
charith@DESKTOP-RBQQSS:~$ |
```

TASK 4

Objective / Skill:

To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines.

Implementation / Work:

```
/*
 * Week 8 - Task 4: Multi-Stage Pipelines with Descriptor Arrays
 * Objective: To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes,
 *            Coordinate Execution, Cleanup Resources, Test Long Pipelines.
 */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define NUM_CMDS 3

int main() {
    printf("=== Week 8 Task 4: Multi-Stage Pipeline (3 Commands) ===\n");
    printf("Executing pipeline: printf \"gamma\\nalpha\\nbeta\\n\" | sort | grep -n \"[a-z]\"\n\n");
    fflush(stdout);

    // Command definitions
    char *cmd0[] = {"printf", "gamma\\nalpha\\nbeta\\n", NULL};
    char *cmd1[] = {"sort", NULL};
    char *cmd2[] = {"grep", "-n", "[a-z]", NULL};
    char **cmds[NUM_CMDS] = {cmd0, cmd1, cmd2};

    int pipes[NUM_CMDS - 1][2];
    for (int i = 0; i < NUM_CMDS - 1; i++) {
        if (pipe(pipes[i]) < 0) {
            perror("pipe allocation failed");
            return 1;
        }
    }

    pid_t pids[NUM_CMDS];
    for (int i = 0; i < NUM_CMDS; i++) {
        pids[i] = fork();
        if (pids[i] == 0) {
            // Child process i
            if (i > 0) dup2(pipes[i - 1][0], STDIN_FILENO);
            if (i < NUM_CMDS - 1) dup2(pipes[i][1], STDOUT_FILENO);
```

```
            // Close all descriptor array handles
            for (int j = 0; j < NUM_CMDS - 1; j++) {
                close(pipes[j][0]);
                close(pipes[j][1]);
            }

            execvp(cmds[i][0], cmds[i]);
            perror("execvp failed");
            exit(1);
        }
    }

    // Parent: Close all descriptor array handles
    for (int j = 0; j < NUM_CMDS - 1; j++) {
        close(pipes[j][0]);
        close(pipes[j][1]);
    }

    // Coordinate execution and wait for all children
    for (int i = 0; i < NUM_CMDS; i++) {
        int status;
        waitpid(pids[i], &status, 0);
    }

    printf("\nCoordinated execution of 3-stage pipeline completed with zero descriptor leaks!\n");
    return 0;
}
```

Output:

```
charith@DESKTOP-RBQQSS:~$ nano s8.4.c
charith@DESKTOP-RBQQSS:~$ gcc s8.4.c -o s8.4
charith@DESKTOP-RBQQSS:~$ ./s8.4
=== Week 8 Task 4: Multi-Stage Pipeline (3 Commands) ===
Executing pipeline: printf "gamma\nalpha\nbeta\n" | sort | grep -n "[a-z]"

1:alpha
2:beta
3:gamma

Coordinated execution of 3-stage pipeline completed with zero descriptor leaks!
charith@DESKTOP-RBQQSS:~$ |
```